

```

from numpy import arange, dtype
from numpy import mean
from numpy import ones
from numpy.random import randn
from numpy.random import randint
from keras.datasets import mnist
from keras.layers import Input, Dense,
    Flatten
from keras.layers import LeakyReLU
from keras.layers import BatchNormalization
from keras.initializers

```

```

def wasserstein_loss(y_true, y_pred):
    return tf.reduce_mean(y_true - y_pred)

```

```

def load_real_sample():
    (train_x, train_y), (_, _) = load_data()
    selected_ix = train_y == 5
    x = train_x[selected_ix]
    x = expand_dims(x, axis=-1)
    x = x.astype('float32')
    x = (x - 127.5) / 127.5
    return x, y

```

```

def generate_real_sample(dataset, n_samples):
    ix = randint(0, dataset.shape[0], n_samples)

```

```
y = ones((n_samples, 1))  
return x, y
```

```
class ClipConstraint(Constraint):
```

```
    def __init__(self, clip_values):
```

```
        self.clip_value = clip_value
```

```
    def __call__(self, weights):
```

```
        return backend.clip(weights, -self  
                             .clip_values, self.clip_values)
```

```
def define_vitc(in_shape=(28,28,1)):
```

```
    init = RandomNormal(stddev=0.02)
```

```
    wnst = ClipConstraint(stddev=0.02)
```

```
    model = ClipConstraint(0.01)
```

```
    model = Sequential()
```

```
        padding='same', kernel_initializer  
        = init, kernel_constraint=wnst)
```

```
    model.add(BatchNormalization())
```

```
    model.add(LeakyReLU(alpha=0.2))
```

```
    model.add(Flatten())
```

```
    model.add(Dense(1))
```

```
    opt = RMSprop(learning_rate=0.00005)
```

```
    model.compile(loss=waterstein_loss,  
                  optimizer=opt)
```

```
    return model
```

```
def define_generator(latent_dim):
```

```
    init = RandomNormal(stddev=0.03)
```

```
    model = Sequential()
```

```

model.add(Dense(n_nodes, kernel_initializer=
    = init, input_dim=latent_dim))
model.add(LeakyReLU(alpha=0.2))
model.add(Reshape((7,7,128)))
model.add(Conv2D(128,(4,4),
    strides=(2,2) padding='same'))
model.add(BatchNormalization())
model.add(LeakyReLU(alpha=0.2))
model.add(Conv2D(1,(7,7) activation=
    'tanh', padding='same'))
return model

```

```

def define_gan(generator, critic)
    for layer in critic_layers:
        if not isinstance(layer, BatchNormalization):
            layer.trainable = False
    model = Sequential()
    model.add(generator)
    model.add(critic)
    opt = RMSProp(learning_rate=0.0005)
    model.compile(loss=wasserstein_loss,
        optimizer=opt)
    return model.

```

```

def generate_latent_points(latent_dim,
    n_samples):
    x_input = random(latent_dim*n_
        samples)
    x_input = x_input.reshape((n_samples,
        latent_dim))
    return x_input

```



```

def generate_fake_samples(generator,
                           latent_dim, n_epochs=10)
    bat_per_epo = int(dataset.shape[0] /
                       n_epochs)
    n_steps = bat_per_epo * n_epochs
    half_batch = int(n_steps / 2)
    c1_hist, c2_hist, g_hist = dict(),
                                dict(), dict()
    for i in range(n_steps):
        c1_tmp, c2_tmp = dict(), dict()
        for i in range(half_batch):
            x_real, y_real = generate_real_sample(
                dataset, half_batch)
            c1_loss1 = c_model.train_on_batch(
                (x_real, y_real))
            c1_loss2 = c_model.train_on_batch(
                (x_real, y_fake))
            c1_hist.append(mean(c1_loss1))
            c2_hist.append(mean(c2_loss2))
        x_gen = generate_latent_points(latent_dim,
                                       n_batch)
        y_gen = ones(n_batch, 1)
        g_loss = g_model.train_on_batch(
            (x_gen, y_gen))
        g_hist.append(g_loss)
    print(' > %d, c1 = %.3f, c2 = %.3f,
          g = %.3f' % (i, c1, c2, g))

```

```

c1_hist[1-hist1-1], c2_hist[1-hist1-1], g_hist[1-hist1-1])
if (1+1)%bat_res_aps == 0:
    summarize_performance(1, g_model,

```

```

plot_history(c1_hist, c2_hist,
              g_hist)

```

```

def summarize_performance(step, g_model,
                           latent_dim, n_samples=100):
    x, _ = generator_take_samples(g_model,
                                   latent_dim, n_samples)
    x = (x+1)/2.0

```

```

    for i in range(10*10):

```

```

        pyplot.subplot(10, 10, 1+i)

```

```

        pyplot.axis('off')

```

```

        pyplot.imshow(x[i, :, :, 0], cmap

```

```

                        = 'gray-r')

```

```

        filename = 'plot_%.04d.png' %
                    (step+1)

```

```

        pyplot.savefig(filename)

```

```

    pyplot.close()

```

```

def plot_history(d1_hist, d2_hist,

```

```

                  g_hist):
    pyplot.plot(d1_hist, d2_hist, g_hist):

```

```

    pyplot.plot(d2_hist, label='crit_val')

```

```

    pyplot.plot(g_hist, label='gen')

```

```

    pyplot.legend()

```

```

    pyplot.savefig('line_plot_loss.png')

```

latent_dim = 50

critic = define_critic()

generator = define_generator(latent_dim)

gan_model = define_generator, critic)

dataset = load_real_samples()

print(dataset.shape)

train(generator, critic gan_model)